
ICCAD 2026 CAD Contest Problem D

Timing Fixing by Useful Skew

Aaron Wu

Department of Electrical Engineering
National Taiwan University
b13901011@ntu.edu.tw

Yucheng Liu

Department of Electrical Engineering
National Taiwan University
b13901104@ntu.edu.tw

Sheng-Yu Cheng

Department of Electrical Engineering
National Taiwan University
b13901088@ntu.edu.tw

Hong-Bin Lai

Department of Electrical Engineering
National Taiwan University
b13901078@ntu.edu.tw

Abstract

1 Introduction

ICCAD Contest 2026 Problem D asks us to improve such violations by modifying the clock tree rather than changing the data-path logic. The available operations are local clock-tree edits: inserting a new buffer, removing an inserted buffer, or resizing an existing buffer. These operations change clock arrival times at flip-flops and therefore change the skew between launch and capture registers.

The key opportunity is useful skew. A nonzero clock skew is not always harmful: delaying the capture clock can give a setup-critical data path more time, while delaying the launch clock can help avoid hold violations by postponing data launch. However, each clock-tree edit may affect many data paths at once. Improving one endpoint may degrade another path, and adding buffers also increases area. The optimization problem is therefore not only to remove timing violations, but to balance setup improvement, hold improvement, and area cost.

In this project, we model the clock tree as a mutable state and search over a space of legal buffer insertion, removal, and resizing operations. Our solver uses candidate policies to decide which moves should be evaluated, and acceptance policies to decide which evaluated move should be applied. This framework allows us to compare greedy search, simulated annealing, iterated simulated annealing, two-step timing repair, and tabu search under a common clock-tree timing model. The experimental discussion focuses on how these policies trade off setup slack, hold slack, and area.

2 Problem Formulation

2.1 Slack, Skew, and Score

A data path connects a launch flip-flop to a capture flip-flop through combinational logic. Let D be the data-path delay, T_{clk} be the clock period, T_{setup} be the setup requirement, and T_{hold} be the hold requirement. Let C_{launch} and C_{capture} denote the clock arrival times from the clock source to the launch and capture flip-flops. We define clock skew as

$$\text{skew} = C_{\text{capture}} - C_{\text{launch}}. \quad (1)$$

With this convention, the setup and hold slacks used in the presentation are

$$\text{setup_slack} = T_{\text{clk}} - T_{\text{setup}} - D + \text{skew}, \quad (2)$$

$$\text{hold_slack} = D - T_{\text{hold}} - \text{skew}. \quad (3)$$

A negative setup slack means that data arrives too late at the capture flip-flop. A negative hold slack means that data arrives too early and may overwrite the previous value. Therefore, a setup violation can often be improved by increasing the capture clock delay, while a hold violation can often be improved by increasing the launch clock delay.

The objective combines timing and area. Timing quality is measured by total negative slack (TNS) and worst negative slack (WNS) under the setup-oriented SS corner and the hold-oriented FF corner. Area is the total area of the clock-tree

buffers. Given a baseline metric value x_0 and a final value x , we use the relative improvement form

$$I(x, x_0) = 1 - \frac{x}{x_0}, \quad (4)$$

when x_0 is nonzero. This rewards reducing negative timing metrics toward zero and rewards reducing area below the baseline. The implemented score is

$$\begin{aligned} S_{\text{setup}} &= I(\text{TNS}_{SS}, \text{TNS}_{SS}^0) + I(\text{WNS}_{SS}, \text{WNS}_{SS}^0), \\ S_{\text{hold}} &= I(\text{TNS}_{FF}, \text{TNS}_{FF}^0) + I(\text{WNS}_{FF}, \text{WNS}_{FF}^0), \\ S_{\text{area}} &= 1 - \frac{\text{Area}}{\text{Area}^0}, \\ S &= \alpha S_{\text{setup}} + \beta S_{\text{hold}} + \gamma S_{\text{area}}, \\ \alpha &= 0.5, \quad \beta = 0.25, \quad \gamma = 0.25. \end{aligned} \quad (5)$$

This formula explains the main trade-off in the problem: aggressive buffer insertion may improve setup or hold slack, but the same move can reduce the area component and may also hurt timing on other paths. The actual parameter is not given; therefore, we set parameters based on the importance for evaluation.

2.2 Optimization Moves

The state space consists of all clock trees reachable from the input tree by legal edits. A state records which buffers are present, the cell type of each buffer, and the parent-child connectivity of the clock tree. Original contest nodes remain part of the design, while buffers inserted by the optimizer may be removed later.

Our optimizer uses three move types. First, an *insert* move places a new buffer on an existing clock-tree edge. This increases the clock delay of the downstream subtree and can be used to adjust useful skew, but it also increases area. Second, a *remove* move deletes a buffer that was inserted by the optimizer and reconnects the surrounding tree. This can recover area, but it may undo a previous timing repair. Third, a *resize* move changes the cell type of an existing buffer. Resizing gives a finer way to increase or decrease clock delay and area without changing the clock-tree topology.

Because the number of possible edit sequences is large, the solver cannot enumerate the full state space. Instead, each iteration generates a smaller set of candidate moves and evaluates their impact on the current timing and score. The rest of the report describes how we construct these candidate sets and how different search policies decide which move to accept.

3 Methodology

3.1 Model for Optimization

Clock Tree. The clock tree is the primary data structure in our optimization framework. It consists of the clock source as the root, buffers as internal nodes, and flip-flops as leaves. The supported operations include buffer insertion, buffer removal, and buffer resizing.

Data Path Graph. The data path graph stores the launch flip-flop, capture flip-flop, and data-path delay information. This information is required to evaluate whether a clock-tree modification improves timing slack.

Timing State. The timing state maintains timing and area metrics, including setup slack, hold slack, total negative slack (TNS), worst negative slack (WNS), and area. These metrics are updated and queried during optimization.

Design Philosophy.

Minimize Re-computation. Recomputing timing information for the entire clock tree after every modification is computationally expensive. Therefore, we employ an incremental timing cache that updates only the affected paths instead of the entire structure.

Recoverability. The optimization process must support undo operations so that a modification can be reverted efficiently when the move is rejected.

3.2 Candidate Policies

Violation Path. We check all data paths and find out those whose slack is negative and smallest, which means the data path violate timing constrain. If setup constrain is violated, the buffers near capture flip-flop is modified. Otherwise, if hold constrain is violated, the buffers near launch flip-flop is modified

Upstream Window. We look for violation endpoint (i.e. capture flip-flop) and explore several buffers toward upstream of clock tree. The policy adds more candidate for optimization, but also more time consuming.

Critical Endpoint. We define *criticality* as an evaluation metric for how much a flip-flop worsen total negative slack. Negative setup slack is added to violation path’s capture flip-flop’s criticality, and negative hold slack is added to launch flip-flop’s criticality, respectively. We try to modify the flip-flop with largest criticality to optimize the violation on more than one data path.

3.3 Accept Policies

Greedy Approach. Based on chosen candidate policy, we apply all generated candidate operations to current clock tree and calculate $\Delta Score$. The strategy accept only operations with $\Delta Score > 0$ and the largest $\Delta Score$.

Two Step Optimization. It’s similar to greedy approach but with two stage focusing on different objective. In the first stage, we try to optimize timing constrain and neglect the penalty caused by area increment. In the second stage, we try to remove and resize buffers to reduce area penalty with timing performance maintained.

Simulated Annealing. The strategy randomly come up with an move and apply the move on current clock tree. If $\Delta Score > 0$ then the move is accepted, otherwise with a certain probability determined by the temperature, this move will be accepted. When the temperature is low, the strategy act like greedy approach. *Iterated Simulated Annealing.* The strategy take turns doing standard simulated annealing step and greedy step, trying to explore more possible moves.

Tabu Search. In each step of the strategy, we pick the best move among all candidates greedily. If the move is non-tabu move, then we directly accept the move, otherwise only if the move creates new global optimal will the move be accepted.

4 Experimental Results and Discussion

4.1 Overall Comparison

4.2 Setup, Hold, and Area Trade-off

4.3 Policy-Level Discussion

4.4 Per-Testcase Behavior

5 Conclusion and Future Work

Author Contributions

Generative AI Usage Disclosure

We used generative AI tools during the development of this project. In particular, OpenAI Codex and Cursor was used extensively to assist with source-code implementation, including parser construction, clock-tree data structures, timing evaluation routines, optimization-loop implementation, test scripts, and refactoring. Approximately 80% of the submitted source code was initially generated or modified with the assistance of Codex or Cursor.

The algorithmic ideas, including the useful-skew optimization strategy, candidate generation policies, acceptance strategies, timing-area trade-off considerations, experimental design, and final optimizer selection, were designed and evaluated by the team members. All AI-generated code was manually reviewed, integrated, debugged, and tested by the team. The reported experimental results were obtained by executing our implemented solver on the test cases; no experimental data or performance numbers were generated by AI tools.

We also used ChatGPT/Codex to assist with report organization, wording refinement, and documentation polishing. The final technical content, claims, results, and conclusions were checked and approved by the team.

References